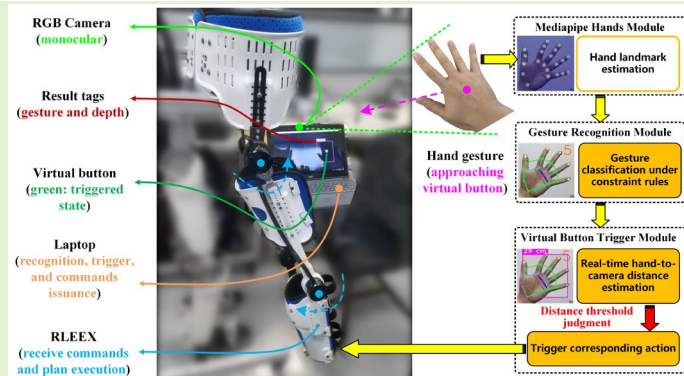


Vision-Based Gesture-Driven Control Method for Rehabilitation Lower Extremity Exoskeleton Robots

Shuang Qiu, Yingjie Liu, Chen Wang, Jing Zhang, Wenhao Xian, and Changbing Chen

Abstract—Recent advances in Rehabilitation Lower Extremity Robotic Exoskeletons (RLEEX) have driven significant improvements in Human Robot Interaction (HRI) methods, including traditional physical interaction and bioelectrical signal approaches. However, most existing HRI methods rely on contact-based interaction, which introduces challenges such as operational complexity, susceptibility to interference, risks associated with implantable devices, and comfort limitations. These issues hinder intuitive and natural interaction, potentially reducing patient motivation during rehabilitation. To address these limitations, this study proposes a novel non-contact gesture interaction control method for RLEEX, utilizing RGB monocular camera depth estimation. The method comprises three core steps: key points detection, gesture recognition, and distance assessment, integrating gesture information with augmented reality triggering technology to control RLEEX gait movements. Experimental results demonstrate that this approach effectively mitigates issues of poor comfort, low reliability, and high latency in contact-based HRI. Specifically, the non-contact method achieves a gesture-controlled exoskeleton motion accuracy of 94.11% and an average system response time of 0.615 seconds. This innovative non-contact HRI framework represents a pioneering advancement in RLEEX control interactions, offering a promising direction for future research and development in the field.

Index Terms—Rehabilitation lower extremity robotic exoskeletons; Human robot interaction; Gesture recognition; Distance estimation; Augmented reality



I. INTRODUCTION

EXOSKELETON robots are wearable devices designed to be worn by humans and moves along with the limbs to perform tasks. These systems primarily consist of an external framework worn on the body and a powered actuation system. Their distinguishing feature lies in leveraging the powered system to assist the movement of human joints and muscles, thereby enhancing, extending, compensating for, or replacing the functional capabilities of human limbs [1]. Applications include performance augmentation, increased endurance, and mobility assistance for individuals with disabilities or the elderly. Unlike traditional rehabilitation devices, which are often limited to use in controlled environments such as hospitals, robotic exoskeleton systems can be used in a broader range of settings. Although extensive research has been conducted on robotic exoskeletons, their control remains a

significant challenge [2], [3], [4]. Special attention must be paid to ensuring user-friendly interaction between the exoskeleton and humans in most application scenarios [5], [6], [7]. This is particularly critical for rehabilitation contexts, such as gait retraining for individuals with Spinal Cord Injuries (SCI) or stroke patients [8].

In lower limb rehabilitation, active patient participation in leg control is vital, requiring gradual training to master normal gait through assistance. Identifying the patient's active intent is critical for effective Human Robot Interaction (HRI) and rehabilitation. Accurate intent recognition ensures training efficacy, relying on reliable signal sources and robust HRI to align exoskeleton movements with patient intent.

Current methods often use physical interaction tools like mechanical sensors (e.g., Inertial Measurement Units (IMU), angular encoders, foot pressure sensors) to capture intent signals [9]. However, these sensors detect movements only after they occur, causing delays, and their bulkiness limits practical use. Traditional Rehabilitation Lower Extremity Robotic Exoskeletons (RLEEX) rely on buttons, joysticks, or wrist controls (e.g., Atalante's button controller [10], Rex's waist mounted joystick [11], ReWalk's wristwatch [12]). While stable for basic mode switching, increasing motion mode diversity complicates these systems, raising implementation challenges and user burden.

For paralyzed patients, accurately recognizing complex

Manuscript received: xxx xx, 2025; Revised: xxx xx, 2025; Accepted: xxx xx, 2025. (Corresponding author: Yingjie Liu.)

Shuang Qiu, Yingjie Liu, Wenhao Xian, and Changbing Chen are with the Emergency science research institute, China Coal Research Institute, Beijing, China. Shuang Qiu is also with the Beihang University, Beijing, China. (email: qiushuang@buaa.edu.cn, liuyingjie_ccri@126.com, xwh17@tsinghua.org.cn, stark_ccb@163.com).

Chen Wang, and Jing Zhang are with the Zhejiang Sci-Tech University, Hangzhou, China. (email: wangc@zstu.edu.cn, 2024210702015@mails.zstu.edu.cn).

movement intents is especially critical, yet weakened lower-limb muscles limit contact-based methods (e.g., force/bioelectrical feedback). Unaffected upper limbs, with stable motor signals, have thus become a key focus for rehabilitation innovation.

To help lower-limb paralysis patients overcome the challenges of using lower-limb electromyographic (EMG) signals, researchers have attempted to utilize EMG signals from the unaffected upper limbs for human motion intention recognition. For example, the MINDWALKER exoskeleton predicts lower-limb movement by leveraging the correlation between arm and leg movements during walking, using upper-limb EMG signals to forecast the motion trajectory of the thighs, calves, and feet [13]. Furthermore, EMG signals from upper-limb gestures can provide the necessary motion intentions for multi-mode control of RLEEX, allowing the user to control the exoskeleton through gestures [14]. However, as physiological signals, EMG signals are susceptible to interference from factors such as noise [15], sweat [16], and skin impedance [17]. There are also inter individual differences and intra individual variations [18], which limit the widespread application of EMG. Similarly, other bioelectrical signals, such as electroencephalography (EEG), electrooculography (EOG), and electrocardiography (ECG), face analogous challenges [19], including risks associated with implantation, low signal-to-noise ratios, susceptibility to interference, individual variability, limited signal bandwidth, and difficulty in identifying multiple intentions under complex movement patterns [20]. Moreover, the hardware and algorithmic complexities involved in the acquisition and processing of these signals add to the overall cost.

The aforementioned HRI technologies, whether based on IMUs, angle sensors, force sensors, controllers, joysticks, wristwatches, or bioelectrode wearable devices, are all contact-based. These systems often exhibit some limitations in terms of comfort, robustness, and sensitivity, and their ability to accurately reflect human motion intentions is also constrained [21]. As a result, non-contact interaction technologies, such as computer vision, have emerged as a promising new direction [22]. Computer vision, by mimicking biological vision, enables machines to perceive their surrounding environment. In recent years, it has developed rapidly [23], [24], offering a low-cost and easily deployable solution. Similarly, computer vision can recognize users' motion intentions and actions, allowing exoskeleton robots to synchronize with the user's movements and providing a more natural and intuitive interaction experience. However, most visual methods depend on deep learning neural networks, which face challenges in complex scenes: lighting variations and background complexity degrade image quality and intent recognition reliability [25], complicating reliable motion capture in dynamic environments.

To address issues such as poor comfort, low reliability, and high latency in the HRI control of RLEEX, this paper proposes a non-contact HRI control method based on computer vision. The method utilizes a three-step fusion algorithm, called Key points detection Gesture classification and Distance triggering Fusion Algorithm (KG DFA), based on an RGB monocular camera. The algorithm first employs the Mediapipe Hands algorithm [26] for hand key points detection, optimizing the key points connection logic. Then, using the improved key

points connections, a two-layer recognition convex structure is constructed to identify different hand gestures. For flexion gestures, the algorithm calculates the two-dimensional angles formed by key points connections, setting thresholds to classify the gestures and improve recognition accuracy. Finally, the method uses monocular RGB vision to trigger virtual buttons, controlling the exoskeleton to execute corresponding gait actions, thus enhancing interaction safety. In addition, a Finite State Machine (FSM) is designed to ensure the safe transition between gait modes. This innovative HRI control method, based on visual detection, recognition, and augmented reality triggering, offers potential applications in fields such as rehabilitation assessment [27], [28].

A. Contribution

The main contributions of our work are summarized as follows:

1) In light of the challenges posed by the weakened muscle strength of lower-limb paralysis patients and the limited number of recognizable patterns in brainwave signals, this study focuses on gesture-based motion intention recognition. It proposes a novel HRI method, called KG DFA, and establishes a HRI interface for a RLEEX platform using an RGB monocular camera.

2) The developed HRI control system, KG DFA, relies on hand key points detection, a two-layer recognition convex framework, and constraints on the two-dimensional joint angles of fingers to interpret gestures. This approach addresses the challenge of recognizing multiple intentions under complex motion patterns at a relatively low cost.

3) KG DFA employs distance estimation using a low-cost and widely applicable monocular RGB camera to enable virtual button triggering, thereby avoiding the complex computations and potential failures associated with using depth cameras to measure hand distance. On one hand, this approach simplifies the system's signal processing complexity and improves real-time performance. On the other hand, it effectively addresses the low reliability often associated with visual recognition-based interactions, thereby ensuring the safety of HRI.

4) The gesture-controlled RLEEX achieved a motion execution accuracy of 94.11%, indicating its effectiveness and reliability, making it more feasible for engineering applications. The average response time of the gesture control system is approximately 0.615 seconds, indicating its ability to perform real-time control and execution.

5) FSM was designed for the actions of RLEEX, further enhancing the safe transition of gait motion modes under gesture-based interaction.

B. Organization

The remainder of this paper is organized as follows. Section II reviews related work on gesture recognition and interaction control. The proposed KG DFA and the FSM transition strategy for exoskeleton gait movements are detailed in Section III. Experimental evaluation results are presented in Section IV. Finally, conclusions and directions for future work are discussed in Section V.

II. RELATED WORK

Gesture recognition is a popular research topic in the HRI of RLEEX, playing a crucial role in the interaction between humans and exoskeletons [29]. Two of the most widely used gesture recognition techniques are surface electromyography (sEMG)-based and vision-based recognition.

sEMG-based methods process sEMG signals to interpret gestures [30], but their generalizability is limited by factors like electrode displacement, muscle fatigue, individual signal variations, and time-dependent muscle changes, restricting widespread use. In contrast, vision-based recognition uses visual data from sensors and deep learning neural networks to classify gestures from large, diverse datasets, offering strong generalizability across users. As a non-contact approach, it avoids discomfort caused by sensors touching the skin, making HRI more natural and comfortable.

Notable advancements include: Xu *et al.* [31] developing a hybrid Convolutional Neural Network (CNN) framework for accurate multi-hand detection in cluttered backgrounds. Ibanez *et al.* [32] introducing EasyGR, a Machine Learning (ML) tool enabling gesture customization without advanced ML expertise. And Mujahid *et al.* [33] presented a lightweight gesture recognition model based on YOLOv3 and DarkNet-53 CNN [34], that requires no preprocessing and works reliably in low-resolution, complex environments.

Gesture recognition technology is gaining traction across fields like medicine, construction, and robotics to enhance labor productivity [35], offering safer, more intuitive interaction than button-based controls [36].

Key applications include: Rudd *et al.* [37] integrating gesture control with robotic motion systems for precise position/speed regulation and obstacle avoidance. Neto *et al.* [38] deploying gesture-based HRI system in manufacturing to assist assembly tasks. Mazhar *et al.* [39], [40] developing a real-time system using Kinect depth data, OpenPose skeletal extraction [41], and CNNs to recognize gestures, enabling the robot to perform tasks such as tool exchange between humans and robots. Shi *et al.* [42] proposed a real-time gesture-controlled robotic 3D printing HRI method. This method uses YOLOv5 [43] and Mediapipe algorithms to recognize gestures and translate gesture information into real-time robotic operations. Experimental results demonstrated the effectiveness and reliability of gesture control. Regarding visual gesture recognition for exoskeleton interaction control, Baulig *et al.* [44] evaluated the performance of a robot-controlled, hand-based monocular hand posture estimation in simulation. However, their study did not conduct experiments on real exoskeleton prototypes.

Currently, gesture control technology faces challenges such as low precision, complex operation, and prolonged recognition time. The accuracy of gesture recognition directly impacts the precision of gesture control. This study aims to improve existing gesture recognition algorithms to enhance their accuracy and real-time performance; And introduces augmented reality triggering based on depth estimation of monocular RGB camera, which sets triggering conditions with depth threshold to improve the reliability of interaction; Thus ensuring the quality of gesture-controlled RLEEX.

III. METHODOLOGY

The following statement is made for this paper: 1) All methods were carried out in accordance with relevant guidelines and regulations. 2) All experimental protocols in our paper were approved by the School of Automation Science and Electrical Engineering of Beihang University. 3) Informed consent was obtained from all subjects.

This study focuses on controlling exoskeleton robots to perform specific movements through gesture control. The objective is to obtain digitized gesture results via gesture recognition software and convert them into appropriate control commands. First, we designed and implemented hand key points detection based on MediaPipe Hands to distinguish the interactive hand from the background. On this basis, we improved the key points connection logic to better support gesture recognition and introduced a virtual key point to simplify the overall palm computation. Second, we developed a gesture recognition rule-based algorithm leveraging the improved key points connection logic, enabling the classification of 12 distinct gestures, each corresponding to a specific gait pattern of the exoskeleton. Furthermore, we designed a virtual button system based on monocular depth estimation, which requires both a specific gesture and a press-and-release action of the virtual button to control the execution of the corresponding gait task. This approach ensures reliable HRI while supporting multitasking scenarios. Finally, a FSM gait transition strategy for the exoskeleton gait action is designed to achieve the goal of augmented reality interactive control of the exoskeleton motion mode.

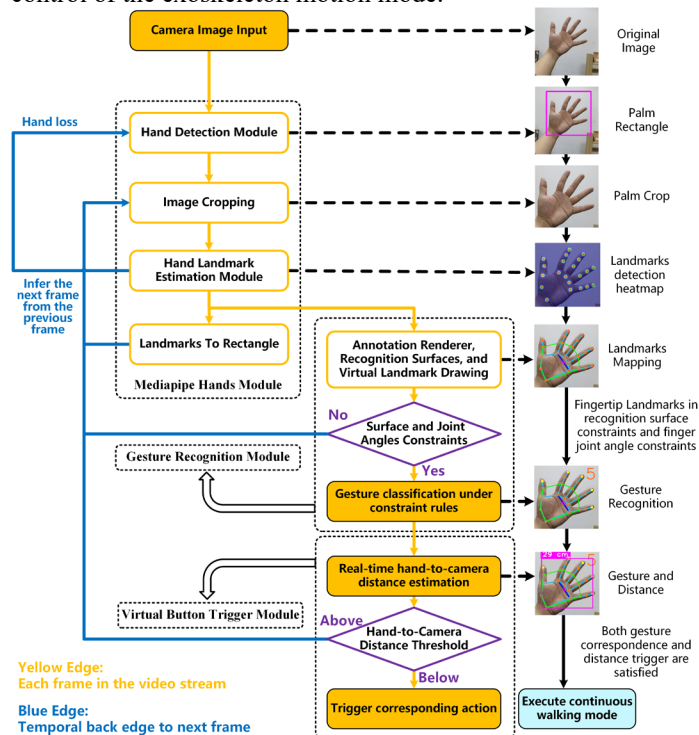


Fig. 1. Flowchart of key points detection-gesture classification and distance triggering fusion algorithm (KG DFA).

The workflow of the proposed KG DFA is illustrated in Fig. 1. The algorithm consists of several preprocessing and analysis steps to ensure high-quality results. First, the raw images are preprocessed through denoising and sharpening to reduce data redundancy and complexity while enhancing the detectability

of useful information within the images. Next, a palm detection module is used to identify the gesture region, cropping out redundant parts to obtain a refined gesture image. Subsequently, a hand key points estimation module is employed to generate joint heatmaps, from which the positions of joint nodes are determined to extract key points data and identify the hand skeleton. Then, a two-layer recognition convex surface, specific key points of the five fingertips, finger flexion degree and two-dimensional joint node constraints of pixel distance ratio are introduced to realize gesture recognition. This workflow automatically crops images and achieves high-precision, high-reliability gesture information recognition. Finally, the real-time perpendicular distance between the palm and the camera is calculated to determine whether to trigger a virtual button, thereby executing the exoskeleton gait task corresponding to the recognized gesture.

A. Mapping Relationship Between Gestures and Exoskeleton Actions

In the context of gesture recognition for controlling exoskeletons, two fundamental prerequisites for effective gesture detection are: 1. the gesture must be recognizable by the camera, and 2. the operator should be able to easily learn, perform, and recall the gesture. Ensuring that gestures are simple to recognize, execute, and replicate is essential for reliable operation. To accommodate the diverse gait modes of the exoskeleton, we customized 12 specific gestures. These include ten numerical gestures (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) and two symbolic gestures (“love” and “rock”). The distribution of these specific gesture categories and their corresponding gait modes is shown in Table I. As illustrated, these gestures are straightforward, systematic, and easy to recognize.

To help the model better learn and understand gesture information, we drew inspiration from the open-source datasets MSRA Hand Tracking database [45] and MSRA Hand Gesture database [46], which include diverse gesture images captured from various angles and orientations. Each hand was annotated with 21 key points, representing landmarks (i.e., the coordinates of points within the images). The annotated dataset was then randomly divided into 80% training data and 20% testing data. After training and evaluation, the final model was obtained.

B. Gesture Recognition Algorithm Based on Hand Key points Detection

This study uses MediaPipe Hands [18] for hand key points detection and tracking, which relies on two core modules: palm detection and hand key points estimation. Finger flexibility complicates direct gesture recognition, so the workflow splits into two steps. First, the palm detection module identifies the palm using its relative rigidity as a stable reference. Once the palm is detected, the hand key points module identifies specific finger key points for accurate gesture recognition. To optimize efficiency, the hand key points module tracks the palm's position using coordinates from the previous frame, reducing redundant palm detection across frames. If the palm's confidence level drops below a threshold, the palm detection module reactivates. Synchronized components—including a real-time rate limiter, image cropping module, key points based

palm position estimation module, and rendering module—ensure high performance. This design accelerates processing speed, enhancing real-time applicability for rapid, accurate hand tracking.

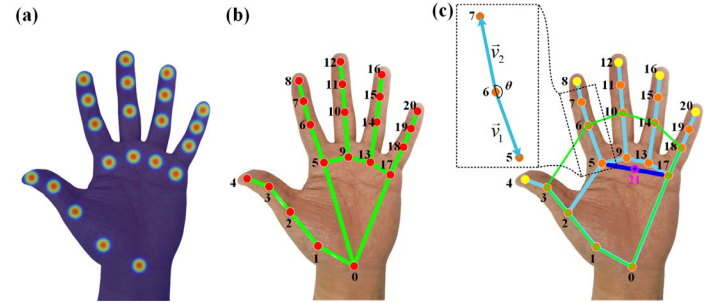


Fig. 2. Identification diagram: (a) Heatmap; (b) MediaPipe skeleton model; (c) Identify convex surfaces, virtual key points, and knuckle angles.

The hand key points estimation module predicts and generates heatmaps for 21 hand key points, from which the coordinates of these key points are extracted. By numbering and connecting these 21 key points in a specific order, a skeletal model of the hand is formed, as illustrated in Fig. 2 (a) and Fig. 2 (b). To construct the palm plane, MediaPipe connects key points 0, 5, 9, 13, and 17 sequentially. However, the resulting area is relatively small, which can lead to misrecognition of various gestures, adversely affecting gesture interpretation and subsequent control tasks.

After designing the gesture task state mapping and implementing hand key points detection and tracking, our goal is to accurately recognize gestures while distinguishing the operator from the background. To achieve this and address the limitation of the small palm region, we improved the key points connection logic based on MediaPipe Hands. Specifically, we connected key point 2 to key point 5 and extended the approximate triangular palm region formed by key points 0, 5, 9, 13, and 17 into a quadrilateral region by incorporating key points 0, 1, 2, 5, 9, 13, and 17. This new region was designated as the inner recognition convex hull. Subsequently, we constructed a larger polygonal area using key points 0, 1, 2, 3, 6, 10, 14, 18, and 17, which was designated as the outer recognition convex hull. The construction of these inner and outer convex hulls facilitates gesture recognition by providing a clear reference. By determining which of the five fingertip key points (4, 8, 12, 16, 20) fall within the inner or outer convex hulls, we designed a set of gesture actions, as specified in Table II. Additionally, for gestures involving finger flexion, we calculated the degree of finger bending by analyzing the two-dimensional joint angles formed by connecting the key points along each finger. Thresholds for the angles between the upper and lower finger joints were set to classify gestures dynamically, thereby further enhancing recognition accuracy. The improved design is illustrated in Fig. 2 (c).

To ensure the accuracy of finger flexion estimation, we set threshold values θ_0 for the angles between the upper and lower joints of the fingers. The vector angles θ_0 between skeletal points of the gesture are calculated using (1):

$$\theta = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \cdot \|\mathbf{v}_2\|}\right) \quad (1)$$

TABLE I

CORRESPONDENCE BETWEEN DIFFERENT GESTURES AND GAIT MOVEMENT PATTERNS

Gesture Classes	Meaning of Gestures	Corresponding Gesture Pictures
Gesture 0	Initialization: Power failure of joint motor or Power on to enable standing	
Gesture 1	Step-by-step walking: Initial right leg step (Starting gait)	
Gesture 2	Step-by-step walking: Walking with left leg (Forward gait)	
Gesture 3	Step-by-step walking: Walking with right leg (Forward gait)	
Gesture 4	Step-by-step walking: End the walk by pulling your legs back (End gait)	
Gesture 5	Continuous walking: Initial left leg step (Starting gait) ^a	
Gesture 6	Return to standing position in continuous walking mode	
Gesture 7	Exoskeleton performs stair climbing gait	
Gesture 8	Exoskeleton performs stair-descent gait	
Gesture 9	Exoskeleton performs obstacle-crossing gait	
Gesture love	Exoskeleton performs sitting action	
Gesture rock	Exoskeleton performs standing action	

^a The left and right legs walk in a cycle and wait for the command to return to the standing position.

The vectors v_1 and v_2 are illustrated in Fig. 2(c). Here, the fingers are defined to be bent for $\theta \leq \theta_0$ and open for $\theta > \theta_0$. During the process of determining whether a finger is bent, it was observed that the thumb, forefinger, and pinky have distinct threshold values compared to the other three fingers. Based on extensive experimentation, the threshold angle for thumb flexion was finalized at 53° . For the forefinger, the flexion angle threshold distinguishing Gesture 0 from Gesture 6 was set at 65° , while the thresholds for Gesture 5, Gesture 9, and other Gestures were determined to be 170° , 120° , and 160° , respectively.

The threshold for pinky flexion was set at 49° , and the thresholds for the other fingers were set at 65° .

In addition, Gestures 6, 7, and 8 require the computation of pixel distances between the fingertip key points 4, 8, 12, and 20 as a constraint. Assuming the pixel coordinates of two distinct fingertip key points in the image are $P^{pi}=[u_1, v_1]^T$ and $P^{pj}=[u_2, v_2]^T$, the pixel distance between them is calculated as follows:

$$P^{pij} = \sqrt{(u_2 - u_1)^2 + (v_2 - v_1)^2} \quad (2)$$

TABLE II
FINGER KEY POINT STATES AND FLEXION THRESHOLDS CORRESPONDING TO DIFFERENT GESTURES

Gesture Classes	Key point status	4	8	12	16	20
		Thumb	Forefinger	Middlefinger	Ringfinger	Pinky
Gesture 0	4, 8, 12, 16, and 20 are all within the inner convex hull	$<53^\circ$	$<65^\circ$	$<65^\circ$	$<65^\circ$	$<49^\circ$
Gesture 1	Only 8 is outside the outer convex hull	$<53^\circ$	$>160^\circ$	$<65^\circ$	$<65^\circ$	$<49^\circ$
Gesture 2	Only 8 and 12 are outside the outer convex hull	$<53^\circ$	$>160^\circ$	$>65^\circ$	$<65^\circ$	$<49^\circ$
Gesture 3	8, 12, and 16 are outside the outer convex hull	$<53^\circ$	$>160^\circ$	$>65^\circ$	$>65^\circ$	$<49^\circ$
Gesture 4	8, 12, 16, and 20 are outside the outer convex hull	$<53^\circ$	$>160^\circ$	$>65^\circ$	$>65^\circ$	$>49^\circ$
Gesture 5	4, 8, 12, 16, and 20 are all outside the outer convex hull	$>53^\circ$	$>170^\circ$	$>65^\circ$	$>65^\circ$	$>49^\circ$
Gesture 6	4 and 20 are outside the outer convex hull and their pixel distance from each other is greater than the set threshold	$>53^\circ$	$<65^\circ$	$<65^\circ$	$<65^\circ$	$>49^\circ$
Gesture 7	4, 8, and 12 are outside the outer convex hull, and their pixel distance ratio from each other is smaller than the set threshold	$>53^\circ$	$>160^\circ$	$>65^\circ$	$<65^\circ$	$<49^\circ$
Gesture 8	4 and 8 are outside the outer convex hull, and their pixel distance from each other is greater than the set threshold	$>53^\circ$	$>160^\circ$	$<65^\circ$	$<65^\circ$	$<49^\circ$
Gesture 9	Only 8 is outside the outer convex hull	$<53^\circ$	$<120^\circ$	$<65^\circ$	$<65^\circ$	$<49^\circ$
Gesture love	4, 8, and 20 are outside the outer convex hull	$>53^\circ$	$>160^\circ$	$<65^\circ$	$<65^\circ$	$>49^\circ$
Gesture rock	Only 8 and 20 are outside the outer recognition convex hull	$<53^\circ$	$>160^\circ$	$<65^\circ$	$<65^\circ$	$>49^\circ$

Through extensive experimentation, the pixel distance constraints for specific gestures were determined as follows: for Gesture 8, the constraint is $P^{p48} > 100$. For Gesture 6, the constraint is $P^{p420} > 100$. For Gesture 7, a proportional constraint was established based on the distances between key points 4

and 12, as well as 8 and 12:

$$\frac{P^{p412}}{P^{p812}} < 2 \quad (3)$$

Table II presents the thresholds for gesture flexion levels,

defined based on the two-layer recognition convex hull and two-dimensional joint angle constraints for finger key points. These thresholds enable the gesture recognition software to accurately detect which layer of the recognition convex hull each fingertip key point is located in. Additionally, it identifies the corresponding gesture by analyzing the degree of finger flexion and the pixel distance ratios between fingertip key points.

After implementing the gesture recognition algorithm, our next objective shifted toward ensuring reliable gesture operations while maintaining smooth interactions and a wide range of interaction capabilities. To achieve this, we enhanced the MediaPipe Hands framework by introducing a virtual key point, referred to as key point 21. This key point 21 is defined as the midpoint of the line segment connecting key points 5 and 17, representing the center of the palm width, as illustrated in Fig. 2(c). We designate this virtual key point 21 as the anchor point of the palm, which simplifies the computation of all key points on the hand. Furthermore, it serves as the basis for estimating the overall depth of the hand in subsequent steps.

C. Depth Estimation based Method for Augmented Reality Interaction Triggering

We identify 22 key points of the hand on the pixel plane and determine their maximum and minimum coordinates. Using these extrema, we construct a rectangular region representing the detected hand area, referred to as the bounding box. When the distance between the hand and the camera remains below a predefined threshold, we check whether the virtual button we set is located within the bounding box. If it is, the virtual button is considered pressed (its color changes to green). Conversely, if the hand to camera distance exceeds the threshold or the hand's bounding box moves away from the virtual button, the button is released (its color changes to red). This mechanism enables the control of virtual button press-and-release functionality in augmented reality interactions. This approach enhances operational certainty and reliability when triggering specific task commands, serving as a safety mechanism. Compared to traditional mechanical button interactions, both methods ensure stable and reliable signals during mode switching. However, with the increasing diversity of motion modes, mechanical buttons require additional physical buttons to accommodate new needs. This leads to an increase in the number of physical interfaces, adding bulk, weight, and complexity to the system while raising the difficulty of user operation. In contrast, the depth estimation based augmented reality interaction triggering method allows the integration of gestures to correspond to diverse motion modes and task actions. This approach not only ensures reliable HRI but also meets the demands of multitasking scenarios, offering a more streamlined and versatile solution.

The distance between an object and the camera can be obtained using either a binocular camera or a monocular camera. A binocular camera estimates depth by combining baseline, focal length, and disparity through triangulation, enabling it to acquire depth information for any pixel in a video frame with high precision and low error. In contrast, a

monocular camera determines the perpendicular distance from the object to the camera based on the pinhole imaging principle. Although it cannot provide depth information for specific pixels and its accuracy is more affected by changes in object posture, it offers advantages in cost, accessibility, applicability, real-time performance, and stability compared to depth cameras. In close-range hand interaction scenarios, due to the pixel size of the hands on the imaging plane, we can use monocular estimation to determine the distance of the hands.

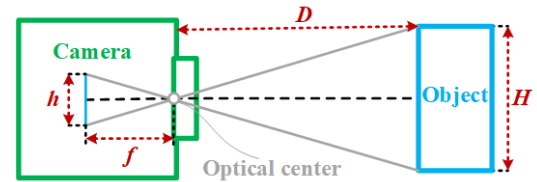


Fig. 3. Monocular pinhole camera ranging diagram.

The similar triangles method is a commonly used approach for distance measurement with monocular cameras. As shown in Fig. 3, suppose we have a target object of height H , placed at a distance D from the camera. After capturing the object and measuring its pixel height h in the image, and given the camera's focal length f , the perpendicular distance from the monocular camera to the object can be determined using the principle of similar triangles:

$$D = \frac{f \cdot H}{h} \quad (4)$$

For distance-triggered interaction using a monocular RGB camera, we utilize the distance between hand key points 5 and 17 as the real-world anchor size for monocular depth estimation. This distance approximately corresponds to the palm width minus the width of one finger (about one quarter of the palm width). By inputting the user's gender and height, the corresponding palm width can be obtained from the Chinese Adult Human Body Dimensions dataset [47], allowing the calculation of the real-world anchor size H . The pixel height h of the line connecting key points 5 and 17 can be obtained in real-time, and with the known camera focal length f , the depth can be computed using (4). This enables interaction based on the calculated depth D . For depth cameras, to maintain consistency with the anchor size used in monocular depth estimation (the line connecting key points 5 and 17) while simplifying the computation, we measure the depth of the virtual key point 21 as a proxy for the overall hand to camera distance.

D. Exoskeleton Actions FSM Transition Strategy

With gesture recognition and distance-trigger algorithms in place, the next step is to design a FSM strategy for controlling exoskeleton movements. FSM is a mathematical model used to describe a finite set of states and the process by which they transition in response to events. It is widely applied in fields such as natural sciences, computer science, automation, and robotics for behavior analysis and modeling [48]. FSM has three core attributes: finite states, only one active state at a time, and transitions triggered by specific conditions. This section leverages FSM to construct the action logic for the exoskeleton,

aiming to achieve two primary objectives: 1) simplify the action control process, enabling continuous state transitions triggered by single actions, and 2) standardize action execution to prevent inappropriate movements caused by misoperation or incorrect gesture recognition.

The FSM model designed in this section is illustrated in Fig. 4. The system comprises eight states: unpowered, standing, sitting, right leg forward, left leg forward, right leg high step, right leg low step, and right leg obstacle-crossing. These states correspond to 12 specific action transitions: initial unpowered to standing (Gesture 0), sitting to standing (Gesture rock), standing to sitting (Gesture love), standing to right leg forward (Gesture 1), right-to-left leg forward (Gesture 2), left-to-right leg forward (Gesture 3), right leg forward to standing (Gesture 4), standing to continuous walking, starting with left leg forward (Gesture 5), continuous walking back to standing (Gesture 6), standing to an ascending stair motion process, starting with right leg high step, and then returning to standing (Gesture 7), standing to a descending stair motion process, starting with right leg low step, and then returning to standing (Gesture 8), standing to an obstacle-crossing motion process, starting with right leg, and then returning to standing (Gesture 9). Each gesture drives hip/knee joints to execute specific trajectories, enabling precise, intuitive exoskeleton control.

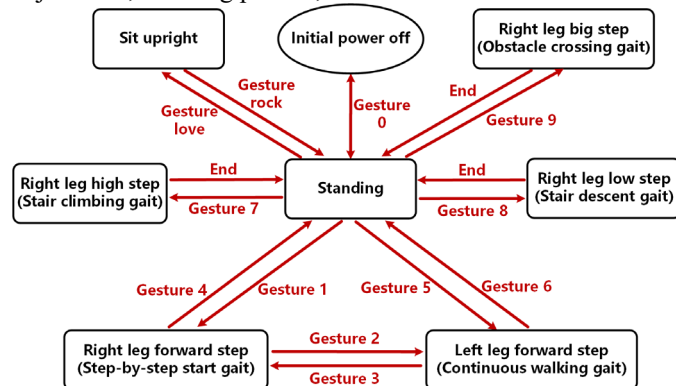


Fig. 4. FSM model of gait switching controlled by gesture in RLEEX.

The transition between different task states requires executing the corresponding gait actions based on gesture commands combined with virtual button trigger conditions. Specifically:

1) The initialization of the exoskeleton, stair ascending, stair descending, and obstacle-crossing tasks are triggered upon receiving user Gestures 0, 7, 8, and 9, respectively. Each gesture triggers the execution of the associated gait trajectory. According to the designed gait trajectories for each task, the exoskeleton returns to the standing state upon completing the respective gait tasks. In addition, When powered on and standing, user gesture 0 triggers power off; a second gesture 0 reactivates standing, cycling between states indefinitely.

2) Users can initiate step-by-step walking task from a standing state by performing Gesture 1 and pressing the virtual button, prompting the exoskeleton to begin the right-leg stepping motion and transition from standing to taking a single step. During the step-by-step walking task, the wearer can use Gestures 2 and 3 to control the exoskeleton for alternate left and right leg walking training. Finally, Gesture 4 triggers the

retraction motion, returning the exoskeleton to the standing state. Alternatively, users can directly trigger a continuous walking task starting with the left leg forward by performing Gesture 5. Gesture 6 allows users to stop walking at any point, initiating a retraction motion to return to the standing state.

3) After the exoskeleton completes initialization and transitions to the standing state, users can trigger the “sit down” action by performing Gesture love. Similarly, the exoskeleton in the sitting position can be returned to the standing state by performing Gesture rock.

To ensure user safety, the exoskeleton temporarily ignores gesture commands while executing a gait task until the action is completed. If the wearer experiences discomfort, an emergency stop button can immediately terminate the exoskeleton's movement. Additionally, a majority voting mechanism is employed to address issues of misoperation caused by fluctuations in gesture recognition, enhancing the robustness of interaction control. The FSM processes gesture commands only after receiving the same gesture input across three consecutive frames. Upon releasing the virtual button, the FSM issues a motion command to the exoskeleton based on the current task state.

IV. EXPERIMENTS AND RESULTS

A. Experimental Platform

The experiments were conducted using the Windows 10 operating system as the training platform, implemented with TensorFlow 2.1.0, Python 3.7, and PyTorch 1.7.0. All training was performed on a desktop computer equipped with an Intel(R) Core(TM) i5-10400F CPU @ 2.90GHz and an NVIDIA GeForce GTX 1080Ti GPU. Testing was carried out on an exoskeleton control platform running on a laptop with an Intel(R) Core(TM) i5-4210U CPU @ 1.70GHz. For visual sensors, the built-in RGB camera of the laptop was used as the monocular camera for the experiments, while the Intel(R) RealSense(TM) D435i depth camera was employed for comparison with the monocular RGB camera. Regarding the exoskeleton, we designed and built a lower-limb rehabilitation exoskeleton featuring Quasi-Direct Drive (QDD) actuated joints to validate the proposed interaction control method.

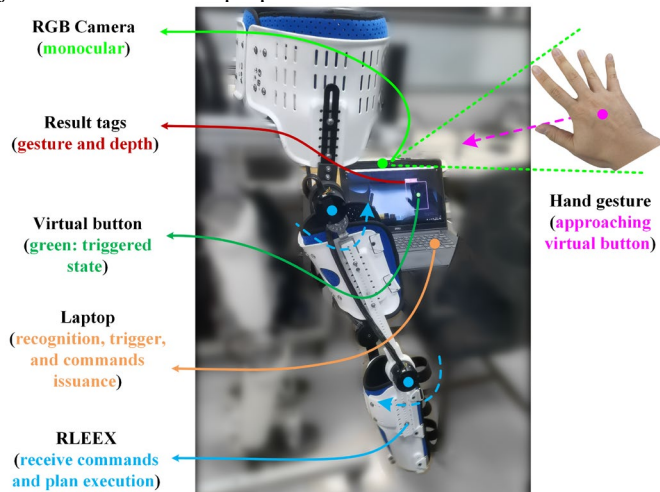


Fig. 5. Physical diagram of the gesture-triggered control RLEEX system.

Fig. 5 illustrates the entire system. In this diagram, the operator controls the RLEEX motion modes using gestures and virtual buttons, addressing the challenges of intent recognition in complex motion modes and the low reliability of visual interaction. We use the webcam built into the laptop to obtain the operator's gesture and distance information. Then, through a host computer program written in Python, the laptop converts the gesture information into the corresponding gait motion information of RLEEX, and uses the distance information as the trigger information for interaction. Finally, the control commands are sent to RLEEX to complete specific gait motions.

B. Accuracy Evaluation of Gesture recognition

The accuracy of gesture recognition is directly related to the stability of the entire system. Before evaluating the overall detection performance, the performance of each individual gesture should be assessed. For hand presence, images with annotated regions are defined as positive samples, while images without annotated regions are defined as negative samples. Based on the classifier model, four possible outcomes can be produced: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). This section evaluates the model performance using five metrics: Precision (Pre), Recall (Re), Intersection over Union (IoU), F1-Score (F1), and Average Precision (AP). These metrics are calculated as follows:

$$Pre = \frac{TP}{TP + FP} \quad (5)$$

$$Re = \frac{TP}{TP + FN} \quad (6)$$

$$IoU = \frac{TP}{TP + FP + FN} \quad (7)$$

$$F1 = \frac{2 \cdot Pre \cdot Re}{Pre + Re} \quad (8)$$

$$AP = \int_0^1 Pre(Re) dRe \quad (9)$$

It can be observed that Precision is the proportion of actual positive samples among those predicted as positive, while Recall is the proportion of actual positive samples correctly predicted as positive. The Intersection over Union (IoU) is the ratio of the intersection of the predicted set and the true set to their union. The F1-Score is the harmonic mean of Precision and Recall, with a maximum value of 1 and a minimum value of 0. Average Precision (AP) combines both Precision and Recall, providing a comprehensive evaluation of their performance. Furthermore, the overall metric for evaluating the performance of an object detection model is the mean Average Precision (mAP), which integrates the average Precision of the model's detections and the average performance across all classes. The calculation method for mAP is as in:

$$mAP = \frac{\sum_{i=1}^k AP_i}{k} \quad (10)$$

In the formula, k represents the total number of gesture categories, and AP_i denotes the accuracy of the i -th category, where i is the sequence number. This paper uses the mAP at an IoU threshold of 0.5 (where samples with an IoU greater than 0.5 with positive samples are considered True Positives, or TP) as the evaluation metric, referred to as mAP@0.5.

Table III presents the evaluation results of the model's predictions for each gesture. It can be observed that Gestures 0 to 6, along with Gestures love and rock, achieved an AP of over 95%, indicating that the KG DFA model has excellent discrimination for these gestures, with Gesture 5 demonstrating the highest level of distinction. The AP for Gestures 7 and 9, however, is slightly lower. This is due to these gestures having different spatial angles in 3D poses, which significantly affect the accuracy of joint skeleton angles and key points distance calculations when projected onto the 2D plane, leading to a decline in recognition precision and other related metrics. Although the evaluation results for these gestures are relatively lower compared to others, their AP remains above 85%, which is sufficient to distinguish and recognize gestures for our gesture recognition and trigger control tasks.

TABLE III
EVALUATION RESULTS OF KG DFA MODEL FOR DIFFERENT GESTURE PREDICTIONS

Gesture Classes	Pre(%)	Re(%)	IoU(%)	F1(%)	AP(%)
Gesture 0	97.92	93.31	96.88	93.13	96.02
Gesture 1	98.43	87.73	98.10	87.75	96.49
Gesture 2	98.99	93.74	93.86	93.63	98.89
Gesture 3	98.94	94.43	97.97	94.01	98.12
Gesture 4	96.71	83.36	95.68	89.14	96.63
Gesture 5	99.08	95.96	96.93	95.16	99.01
Gesture 6	98.97	93.26	98.22	93.21	98.91
Gesture 7	89.01	76.98	88.09	76.81	88.33
Gesture 8	93.71	86.11	95.82	90.31	95.58
Gesture 9	86.78	71.22	85.51	71.27	85.07
Gesture love	98.86	94.72	96.05	94.16	96.87
Gesture rock	99.03	96.09	97.95	95.84	98.02

Bold indicates the best result.

To further validate the superiority of the proposed KG DFA model in recognition tasks, several other algorithm models were selected for comparison with the current model. The comparison models include Faster-RCNN, YOLOv3, SSD, Mobilenet-YOLOv4-lite, and YOLOv5m. We embedded the backbone neural network of the proposed KG DFA model with the comparison models, introducing different features, while maintaining consistent hyperparameter settings during the training phase. In this section: 1) The model complexity is evaluated using the number of parameters (Params). Generally, the smaller the number of parameters, the less storage space the model occupies, making deployment and operation easier. 2) The time taken to process gesture information for a single image (single frame) is used, and the prediction results are assessed by the Unit Speed (US) metric. A smaller US indicates better model performance. 3) Frames Per Second (FPS) is used as a real-time performance evaluation metric. A higher FPS indicates lower latency and better real-time performance.

Table IV lists the relevant parameters for various network models. As shown in the table, compared to other algorithms, the KG DFA model proposed in this paper: 1) Achieves a mAP

(mAP@0.5) of 95.67%, outperforming other models, making it the optimal choice for gesture recognition tasks in exoskeleton interaction scenarios. 2) Although the KG DFA model does not have the smallest number of parameters, the time required to process a single frame of gesture information US is only 0.03s, with an FPS of 34, both of which outperform the other models. This indicates that the speed of providing prediction results is sufficient for real-time detection, with low latency, making it suitable for real-time control during the rehabilitation training process using the exoskeleton.

TABLE IV

COMPARISON OF DIFFERENT MODELS FOR GESTURE RECOGNITION

Network Model	Params(M)	mAP0.5(%)	US(s)	FPS
Faster-RCNN	137.06	90.21	0.92	1
YOLOv3	61.53	83.03	0.25	4
SSD	26.29	84.26	0.12	8
Mobilenet-yolov4-lite	41.01	85.25	0.06	17
YOLOv5m	35.7	87.77	0.17	6
ours	43.12	95.67	0.03	34

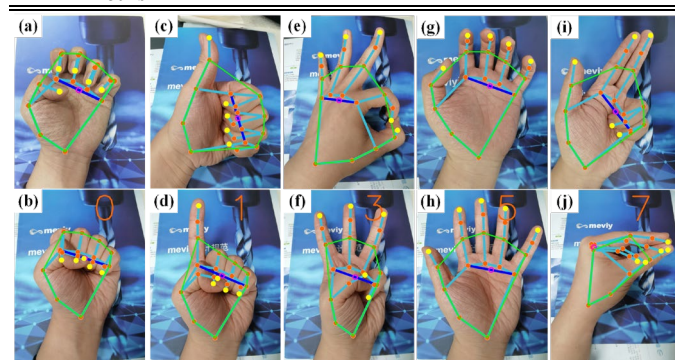


Fig. 6. Confusing gestures: (a) tiger claw hand gesture; (b) defined Gesture 0; (c) good gesture; (d) defined Gesture1; (e) ok gesture; (f) defined Gesture 3; (g) naturally openhand gesture; (h) defined Gesture 5; (i) gun gesture; (j) defined Gesture 7.

Relying solely on object detection for gesture recognition risks errors due to missing constraints on 2D joint nodes (e.g., hand convex surfaces, fingertip key points, finger flexion degree, pixel distances and proportions), leading to misclassification. As shown in Fig. 6 (a), (c), (e), and (g), gestures like “tiger claw”, “good”, “ok”, and “naturally open hand” closely resemble defined Gestures 0, 1, 3, and 5, risking confusion—especially problematic for daily use gestures like “good” and “ok” and should be avoided to ensure they do not conflict with the exoskeleton gesture control.

Our constraint fusion algorithm addresses this by adding targeted constraints: it distinguishes “tiger claw” from Gesture 0 using the two-layer convex surface recognition constraint (Fig. 6 (a) and (b)), separates “good” from Gesture 1 and “ok” from Gesture 3 via incorporating 2D fingertip key point constraints (Fig. 6 (c), (d), (e), (f)), differentiates “naturally open hand” from Gesture 5 using finger flexion degree constraints (Fig. 6 (g), (h)), identifies the “gun” gesture vs. Gesture 7 via applying pixel distance ratio constraints (Fig. 6 (i), (j)), and undefined gestures will not be recognized. These results show the algorithm effectively resolves ambiguities, preventing misclassification.

Therefore, incorporating 2D joint node constraints such as two-layer convex surface recognition, fingertip key points of all five fingers, finger flexion degree, pixel distance, and proportions into the algorithm can significantly improve

gesture recognition accuracy. Integrating these steps into the algorithm enhances the robustness of the gesture recognition software, thereby reducing the misclassification of undefined gestures. The correctly recognized gesture data is then converted into control information for the corresponding exoskeleton robot actions. Furthermore, by adopting the two-stage detection approach from MediaPipe (first detecting the palm, then tracking the finger key points, with palm detection only running when necessary), the real-time control of the exoskeleton movements during rehabilitation training can be effectively achieved.

C. Augmented Reality Triggered Evaluation Validation

When using a monocular RGB camera for depth estimation, we conducted comparative validation with a depth camera. A depth camera can calculate the specific depth of a key point pixel, which is not feasible with a monocular camera. In this study, the depth camera was used to measure the depth of a virtual point (Point 21) as a proxy for the distance between the hand and the camera, instead of relying on multiple key points to determine hand depth. This approach was chosen for three main reasons: 1) Calculating the overall hand depth using the geometric center of all key points increases the likelihood of errors, as the geometric center may not lie within the hand itself. 2) Processing multiple key points imposes significant computational demands on hardware, reducing processing speed and real-time performance. 3) This method aligns with the monocular depth estimation approach, which uses the line segment between key points 5 and 17 as an anchor for scaling. This consistency facilitates a more direct and reliable comparison between the two systems.

For our exoskeleton platform, the camera is installed approximately 50 cm away from the body, 25 cm from the same-side hand, and 60 cm from the opposite-side hand. To prevent accidental activation and ensure safety, the virtual button activation distance must be less than the distance to the nearest hand, which is 25 cm. Considering these factors, we set the activation distance threshold to 20 cm. Consequently, the interaction range for hand-triggered interactions on our platform is 20~60 cm. To evaluate the performance of the two cameras in hand-triggered interaction scenarios, we conducted distance measurement tests from a distance slightly larger than the platform's interaction range, increasing in 5 cm increments from near to far. The actual distances measured using a ruler were used as the reference standard. The errors in distance measurements obtained by the two cameras were compared to provide a comprehensive performance assessment. The measurement data and comparison results are shown in Table V.

The results indicate that the monocular camera is significantly less accurate than the depth camera. The depth camera exhibits the largest error at its minimum range, but the error does not exceed 2%, while errors at other distances are below 1%. In contrast, the monocular camera generally has errors of 2% or higher, with a maximum error of 12.4%. Both cameras perform best at distances of 40~50 cm. At closer distances, accuracy decreases as the distance decreases, and at farther distances, accuracy decreases as the distance increases, indicating a similar trend in depth accuracy between the two cameras. Although the depth camera has higher accuracy, it

cannot cover the interaction range required by our platform. Moreover, within the interaction range, the monocular camera's error remains within 5%, which is acceptable. Therefore, using a monocular camera as the interaction sensor sufficiently meets the requirements for augmented reality-triggered interactions on our platform.

TABLE V

COMPARISON OF DISTANCE ERRORS OBTAINED BY TWO CAMERAS IN HAND-TRIGGERED INTERACTION SCENARIOS

Actual $D(\text{cm})$	Monocular $D(\text{cm})$	Monocular Error(%)	Binocular $D(\text{cm})$	Binocular Error(%)
10	9.3	-7	/	/
15	14.4	-4	/	/
20	19.4	-3	/	/
25	24.3	-2.8	25.4	1.6
30	29.5	-1.6	30.1	0.3
35	34.5	-1.4	34.9	-0.3
40	39.1	-2.25	40.0	0
45	45.3	0.6	45.0	0
50	51.6	3.2	50.0	0
55	57.5	4.5	54.9	-0.2
60	62.9	4.8	59.8	-0.3
65	68.3	5.1	64.7	-0.4
70	78.7	12.4	69.6	-0.6

For the interactive screen output at a resolution of 1280×720 , we used FPS as the metric to evaluate real-time performance. To assess reliability, we adopted the trigger probability—defined as the ratio of successful triggers to the total number of 20 hand posture changes at a specific distance. The results are presented in Table VI.

TABLE VI

COMPARISON OF REAL-TIME PERFORMANCE AND RELIABILITY TEST OF TWO CAMERAS IN HAND-TRIGGERED INTERACTION SCENARIO

Actual $D(\text{cm})$	Monocular FPS	Monocular Trigger Probability(%)	Binocular FPS	Binocular Trigger Probability(%)
10	32	100	19	100
15	31	100	15	100
20	38	80	12	100
25	32	0	11	90
30	33	0	10	30
35	31	0	17	20
40	33	0	16	20
45	37	0	11	10
50	33	0	18	10
55	34	0	11	5
60	32	0	9	5
65	35	0	11	0
70	34	0	10	0

The comparison results indicate that the FPS of the depth camera is concentrated around 10 and 12. This limitation arises from the computational burden of obtaining pixel depth, which reduces hardware processing speed and compromises real-time performance. The highest FPS for the depth camera is 19, observed when the hand occupies most of the frame, as the reduced depth computation workload improves processing efficiency. In contrast, the monocular camera consistently achieves an FPS of 31 or higher, significantly surpassing the maximum FPS of the depth camera and demonstrating excellent real-time performance.

Additionally, we compared specific cases of the two cameras in the hand-triggered interaction scenario, as shown in Fig. 7. Virtual button activation requires hand to camera distance under a threshold (20 cm in this example). Table VI and Fig. 7 (a), (b) show the depth camera fails below 25 cm,

returning a depth value of 0 and falsely activating the button. Moreover, when the hand is at the edge of the video frame, when there is a sudden change in background depth behind the hand, or when the hand posture changes significantly—particularly during flipping or opening/closing motions—the depth camera may also erroneously return a depth value of 0. This occurs because the infrared laser dot array becomes sparse at the edges, and hand flipping motions may cause the laser dots to miss the hand entirely and fall into gaps, resulting in false activations, as illustrated in Fig. 7 (d). This reduces non-contact interaction reliability and raises safety risks. We also observed that the false activation probability decreases with distance—no false triggers occur beyond 65 cm. This behavior is determined by the characteristics of the infrared laser dot array: at close distances, gaps are more likely to occur, while at farther distances, the target object appears smaller in the frame, and the laser dot measurements become more consistent. However, such distances exceed the interaction range of our platform.

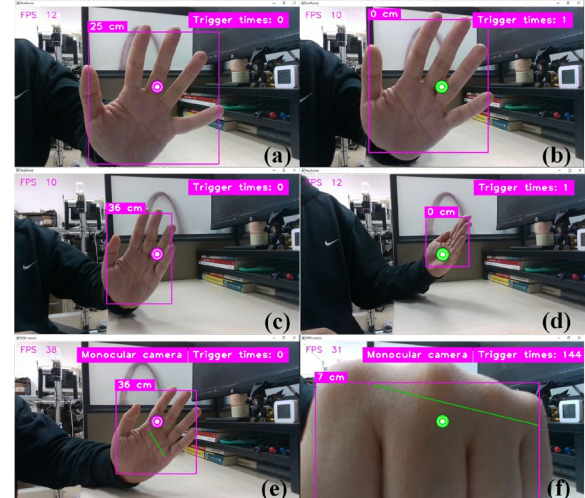


Fig. 7. Special cases of real-time and reliability of two cameras in trigger interaction scenarios (Center virtual button: When it turns green, it means the button is pressed, and when it turns red, it means the button is popped up.): (a) Realsense can measure a minimum depth of 25cm, FPS 12; (b) Realsense returns a measurement value of 0 when it is just less than 25cm, FPS 10; (c) Realsense normally obtains the depth of the palm at 36cm, FPS 10; (d) Realsense incorrectly returns a value of 0 within the measurement range to trigger the virtual button, FPS 12; (e) RGB monocular normal acquisition of palm depth 36cm, FPS 38; (f) RGB monocular can measure a minimum depth of 7cm, FPS 31.

When using a monocular camera, there is a 20% probability of failing to trigger the virtual button during hand posture changes near the threshold. This is because changes in hand posture alter the anchor size in the image. The anchor size is defined as the size of the hand when it is perpendicular to the camera, representing the maximum observable size at that distance. Consequently, any posture change reduces the anchor size, leading to overestimated depth measurements. This characteristic ensures that hand movements, such as flipping, at depths greater than the threshold will not falsely trigger the virtual button, as reflected in the monocular trigger probabilities in Table VI. This guarantees the reliability of virtual button activation during interactions. Furthermore, a comparison of Fig. 7 (b), (d), and (f) reveals that the monocular camera only loses the palm anchor box when the palm depth is less than 7 cm (i.e., when the hand completely

blocks the frame). This demonstrates that the monocular camera has a better dynamic measurement range than the depth camera. Additionally, its depth measurements remain continuously variable without returning erroneous zero values, eliminating the risk of false activations. This enhances the safety and reliability of non-contact hand-controlled exoskeleton interactions.

Of course, monocular camera-based distance measurement also has its challenges, with the most significant being its relatively large errors. This issue is particularly pronounced when the hand is angled relative to the camera, causing the recognized hand area to appear smaller. The reference anchor size also decreases due to the change in perspective, leading to greater measurement inaccuracies. However, in such cases, the monocular camera tends to mistakenly interpret the hand as being farther away, consistent with the pinhole imaging principle of “near objects appear larger, far objects appear smaller”. Therefore, these errors do not affect the functionality of approaching and pressing the virtual button.

D. HRI Experimental Evaluation for Gesture Control of RLEEX

To calculate the total time from performing a gesture to the initiation of the exoskeleton action and to determine the actual success recognition rate, we selected three testers (A, B, and C) to perform 50 independent experiments for each control gesture, as shown in Table VII. All testers were informed of the correct gesture features and the corresponding actions of the rehabilitation exoskeleton robot. Participant A was involved in the network design and training described in this study. To ensure an impartial third-party evaluation, testers B and C were invited and had no prior involvement in any related experiments.

TABLE VII

INFORMATION OF TESTERS PARTICIPATING IN ACTUAL TESTING AND EVALUATION

Testers	Gender	Height (mm)	Hand length (mm)	Hand width (mm)	Palm circumference (mm)
A	Male	1830	195	83	210
B	Female	1560	165	76	179
C	Male	1720	189	89	220

In the practical process of gesture-based interaction control, the RLEEX operation commands exhibit slight delays during execution. These delays arise from communication latencies in transmitting recognized trigger commands from the upper-level controller to the lower-level motor actions, including delays in both Wi-Fi and CAN communication. To accurately measure the time required for gesture-based interaction control, we used recorded videos to ensure that the operator's gestures, the interaction recognition screen on the computer, and the entire exoskeleton system were all effectively captured within the camera's field of view. In the video analysis, the first frame marked the recognition of the operator's gesture, the second frame identified the successful activation of the model (with the virtual button turning from red to green), and the third frame indicated the initiation of the corresponding gait movement by the exoskeleton robot. By conducting multiple measurements and calculating the

average difference between the third and first frames, we determined the Average Response Time (ART), representing the mean time required for a gesture to control RLEEX motion.

The primary focus of the evaluation experiment was to test the number of successfully recognized gestures and the ART of the exoskeleton robot. As shown in Table VIII, the lowest success recognition rate among the three testers was 80.67%, observed for Gesture 9. This recognition rate, combined with the augmented reality trigger conditions, meets the usage requirements. The highest success recognition rate reached 97.33%, demonstrating the robustness of the method and its ability to accurately recognize gestures from different individuals.

TABLE VIII

TEST RESULTS OF ACCURACY AND REAL-TIME PERFORMANCE OF GESTURE CONTROL

Gesture classes	Test numbers	Average successful numbers ^a	Accuracy (%)	ART (s)
Gesture 0	50	44.67(42/48/44)	89.33	0.49
Gesture 1	50	46.33(48/47/44)	92.67	0.50
Gesture 2	50	46.67(46/45/49)	93.33	0.53
Gesture 3	50	46.67(47/45/48)	93.33	0.68
Gesture 4	50	47(48/46/47)	94.00	0.51
Gesture 5	50	48.67(49/48/49)	97.33	0.44
Gesture 6	50	47.67(48/46/49)	95.33	0.46
Gesture 7	50	41.33(44/41/39)	82.67	0.89
Gesture 8	50	43.67(46/42/43)	87.33	0.78
Gesture 9	50	40.33(40/38/43)	80.67	0.99
Gesture love	50	47(46/49/46)	94.00	0.66
Gesture rock	50	48.33(48/49/48)	96.67	0.45

^a The three sets of data in brackets stand for the success counts of each gesture by testers A, B and C.

The ART for the entire process, from initiating a gesture and triggering the virtual button to the robot's response, ranged from 0.99 seconds at its slowest to 0.44 seconds at its fastest. These results strongly validate the effectiveness of the proposed KG DFA model in enabling real-time HRI to control exoskeleton gait actions.

E. Results Analysis

Real-time gesture control requires both accurate gesture recognition and synchronized control of the exoskeleton. Further calculations based on the results in Table VIII show that the average success recognition rate for the three testers was 94.11%, and the total average response time for the system to complete exoskeleton hand gesture control was approximately 0.615 seconds. These findings indicate that the proposed method not only achieves a high success rate and excellent real-time performance but also emphasizes its effectiveness in enabling non-experts to control the robot in real time. The experimental results demonstrate the potential of combining gesture recognition technology, depth estimation based augmented reality techniques, and rehabilitation exoskeleton control technologies.

F. Discussion

The proposed KG DFA model introduces a novel approach to address the challenges encountered in HRI for rehabilitation exoskeletons, including the complex and diverse recognition of human motion intentions and ensuring a reliable, sensitive, and comfortable user experience. Additionally, it enhances the interactive aspect of the rehabilitation training process, making it more engaging for

users. This increased interactivity helps to improve patients' willingness and enthusiasm to participate in their rehabilitation training, encouraging active involvement in the process.

Nevertheless, the method proposed in this study has some limitations: 1) The camera used for gesture recognition has strict requirements regarding the surrounding environment and lighting conditions. 2) The current KG DFA real-time control system experiences latency issues, resulting in slight stuttering during HRI. This prevents rapid adjustments of gait movements through frequent gesture modifications.

Future research should prioritize addressing the aforementioned shortcomings and seek to develop a non-contact real-time control system with broader environmental applicability, lower latency, and greater reliability.

V. CONCLUSION

To address the issues of discomfort, lack of intuitiveness, and unnaturalness associated with contact-based HRI on the RLEEX platform, this paper proposes a non-contact KG DFA real-time interactive control method for RLEEX. This approach not only provides a novel perspective for controlling RLEEX but also offers several advantages over traditional control methods. First, traditional HRI systems typically rely on contact sensors to gather information, which is then fed back to the controller for action. Prolonged interaction with these sensors can lead to physical discomfort for the user. In contrast, the non-contact HRI method proposed in this paper allows users to control the robot remotely, providing a more intuitive, natural, and comfortable experience. Second, traditional visual gesture recognition systems often lack specific trigger conditions, making them prone to false activations, which can reduce the reliability and safety of the interactive control system. The virtual button functionality designed in this study introduces determinism into the interaction process, ensuring the safety of the interaction. Finally, the KG DFA control method proposed here is based on a monocular RGB camera, eliminating the need for a depth camera. This not only reduces costs but also ensures broader applicability. Experimental results further validate the effectiveness and reliability of the RLEEX gesture-based interactive control system.

REFERENCES

- [1] J. L. Pons, *Wearable Robots: Biomechatronic Exoskeletons*. New York, John Wiley & Sons, Ltd, 2008, pp. 1–15.
- [2] P. Huang, Z. Li, M. Zhou, X. Li and M. Cheng, “Fuzzy Enhanced Adaptive Admittance Control of a Wearable Walking Exoskeleton With Step Trajectory Shaping,” *IEEE Trans. Fuzzy Syst.*, vol. 30, no. 6, pp. 1541–1552, June. 2022.
- [3] W. Sun, J. -W. Lin, S. -F. Su, N. Wang and M. J. Er, “Reduced Adaptive Fuzzy Decoupling Control for Lower Limb Exoskeleton,” *IEEE Trans. Cybern.*, vol. 51, no. 3, pp. 1099–1109, March. 2021.
- [4] Z. Li *et al.*, “Human-in-the-Loop Control of a Wearable Lower Limb Exoskeleton for Stable Dynamic Walking,” *IEEE-ASME Trans. Mechatron.*, vol. 26, no. 5, pp. 2700–2711, Oct. 2021.
- [5] U. E. Ogenyi, J. Liu, C. Yang, Z. Ju and H. Liu, “Physical Human–Robot Collaboration: Robotic Systems, Learning Methods, Collaborative Strategies, Sensors, and Actuators,” *IEEE Trans. Cybern.*, vol. 51, no. 4, pp. 1888–1901, April. 2021.
- [6] S. Qiu, Z. Pei, C. Wang, and Z. Tang, “Systematic Review on Wearable Lower Extremity Robotic Exoskeletons for Assisted Locomotion,” *J. Bionic Eng.*, vol. 20, pp. 436–469, 2023.
- [7] A. Martínez, B. Lawson and M. Goldfarb, “A Controller for Guiding Leg Movement During Overground Walking With a Lower Limb Exoskeleton,” *IEEE Trans. Robot.*, vol. 34, no. 1, pp. 183–193, Feb. 2018.
- [8] J. Mehrholz, S. Thomas, C. Werner, J. Kugler, M. Pohl, B. Elsner, “Electromechanical-assisted training for walking after stroke,” *Cochrane Database Syst Rev.*, vol. 10, no. 10, Oct. 2020, Art. no. CD006185.
- [9] L. S. Vargas-Valencia *et al.*, “Sleeve for Knee Angle Monitoring: An IMU-POF Sensor Fusion System,” *IEEE J. Biomed. Health Inform.*, vol. 25, no. 2, pp. 465–474, Feb. 2021.
- [10] T. Gurriet, M. Tucker, A. Duburcq, G. Boeris and A. D. Ames, “Towards Variable Assistance for Lower Body Exoskeletons,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 1, pp. 266–273, Jan. 2020.
- [11] J. L. Contreras-Vidal and R. G. Grossman, “NeuroRex: A clinical neural interface roadmap for EEG-based brain machine interfaces to a lower body robotic exoskeleton,” in *Proc. 35th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, Osaka, Japan, 2013, pp. 1579–1582.
- [12] L. N. Awad, A. Esquenazi, G. E. Francisco, *et al.* “The ReWalk ReStore™ soft robotic exosuit: a multi-site clinical trial of the safety, reliability, and feasibility of exosuit-augmented post-stroke gait rehabilitation,” *J. NeuroEng. Rehabil.*, vol. 17, no. 1, Jun. 2020, Art. no. 80.
- [13] J. Gancet *et al.*, “MINDWALKER: Going one step further with assistive lower limbs exoskeleton for SCI condition subjects,” in *Proc. 4th IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics (BioRob)*, Rome, Italy, 2012, pp. 1794–1800.
- [14] Z. Guo, C. Wang, C. Song, “A real-time stable-control gait switching strategy for lower-limb rehabilitation exoskeleton,” *PloS One*, vol. 15, no. 8, Aug. 2020, Art. no. e0238247.
- [15] J. A. Spanias, E. J. Perreault and L. J. Hargrove, “Detection of and Compensation for EMG Disturbances for Powered Lower Limb Prosthesis Control,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 24, no. 2, pp. 226–234, Feb. 2016.
- [16] J. Cheng, F. L. Wei, C. Li *et al.*, “Position-independent Gesture Recognition using sEMG Signals via Canonical Correlation Analysis,” *Comput. Biol. Med.*, vol. 103, pp. 44–54, 2018.
- [17] J. He *et al.*, “User Adaptation in Long-term, Open-loop Myoelectric Training: Implications for EMG Pattern Recognition in Prosthesis Control,” *J. Neural Eng.*, vol. 12, 2015, Art. no. 046005.
- [18] J. Fan *et al.*, “Surface EMG feature disentanglement for robust pattern recognition,” *Expert Syst. Appl.*, vol. 237, 2024, Art. no. 121224.
- [19] D. P. McMullen *et al.*, “Demonstration of a Semi-Autonomous Hybrid Brain–Machine Interface Using Human Intracranial EEG, Eye Tracking, and Computer Vision to Control a Robotic Upper Limb Prosthetic,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 22, no. 4, pp. 784–796, July 2014.
- [20] H. Dantas, T. C. Hansen, D. J. Warren and V. J. Mathews, “Interpreting Volitional Movement Intent From Biological Signals: A Review,” *IEEE Signal Process. Mag.*, vol. 38, no. 4, pp. 23–33, July 2021.
- [21] M. Iskandar, A. Albu-Schäffer, A. Dietrich, “Intrinsic sense of touch for intuitive physical human-robot interaction,” *Sci. Robot.*, vol. 9, 2024, Art. no. eadn4008.
- [22] Y. Yang *et al.*, “Contactless human–computer interaction system based on three-dimensional holo-graphic display and gesture recognition,” *Appl. Phys. B-Lasers Opt.*, vol. 129, 2023, Art. no. 192.
- [23] C. Wang *et al.*, “StairNetV3: Depth-aware stair modeling using deep learning,” *Vis. Comput.*, vol. 40, pp. 8729–8745, 2024.
- [24] M. Ramanathan *et al.*, “Visual Environment perception for obstacle detection and crossing of lower-limb exoskeletons,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Kyoto, Japan, 2022, pp. 12267–12274.
- [25] B. Lu, B. Bai, X. Zhao, “Vision-based structural displacement measurement under ambient-light changes via deep learning and digital image processing,” *Measurement*, vol. 208, 2023, Art. no. 112480.
- [26] F. Zhang *et al.*, “MediaPipe Hands: On-device real-time hand tracking,” 2020, *arXiv:2006.10214*.
- [27] A. A. Hulleck *et al.*, “BlazePose-Seq2Seq: Leveraging Regular RGB Cameras for Robust Gait Assessment,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 32, pp. 1715–1724, 2024.
- [28] S. Ahn *et al.*, “Multimodal Human Action Recognition for Rehabilitation Exercise of Upper Body for Individuals With Cerebral Palsy,” *IEEE Trans. Syst. Man Cybern. -Syst.*, vol. 54, no. 7, pp. 4164–4174, July 2024.

- [29] M. Yassen, S. S. Jusoh, "A systematic review on hand gesture recognition techniques, challenges and applications," *PeerJ Comput. Sci.*, vol. 5, 2019, Art. no. e218.
- [30] C. Shen *et al.*, "Toward Generalization of sEMG-Based Pattern Recognition: A Novel Feature Extraction for Gesture Recognition," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1-12, 2022, Art. no. 2501412.
- [31] C. Xu *et al.*, "Accurate hand detection from single-color images by reconstructing hand appearances," *Sensors*, vol. 20, 2019, Art. no. 192.
- [32] R. Ibañez *et al.*, "Easy gesture recognition for Kinect," *Adv. Eng. Softw.*, vol. 76, pp. 171-180, 2014.
- [33] A. Mujahid *et al.*, "Real-time hand gesture recognition based on deep learning YOLOv3 model," *Appl. Sci.*, vol. 11, 2021, Art. no. 4164.
- [34] J. Redmon, A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [35] X. Wang, Z. Zhu, "Vision-based framework for automatic interpretation of construction workers' hand gestures," *Autom. Constr.*, vol. 130, 2021, Art. no. 103872.
- [36] M. Y. Shieh *et al.*, "Gesture recognition based human-robot interactive control for robot soccer," *Microsyst. Technol.*, vol. 27, pp. 1175-1186, 2021.
- [37] G. Rudd, L. Daly, F. Cuckov, "Intuitive gesture-based control system with collision avoidance for robotic manipulators," *Ind. Robot.*, vol. 47, pp. 243-251, 2020.
- [38] P. Neto *et al.*, "Gesture-based human-robot interaction for human assistance in manufacturing," *Int. J. Adv. Manuf. Technol.*, vol. 101, pp. 119-135, 2019.
- [39] O. Mazhar *et al.*, "Towards Real-Time Physical Human-Robot Interaction Using Skeleton Information and Hand Gestures," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Madrid, Spain, 2018, pp. 1-6.
- [40] O. Mazhar *et al.*, "A real-time human-robot interaction framework with robust background invariant hand gesture detection," *Robot. Comput.-Integr. Manuf.*, vol. 60, pp. 34-48, 2019.
- [41] Z. Cao *et al.*, "OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172-186, 1 Jan. 2021.
- [42] X. Shi *et al.*, "Research on human-robot interaction for robotic spatial 3D printing based on real-time hand gesture control," *Robot. Comput.-Integr. Manuf.*, vol. 89, 2024, Art. no. 102788.
- [43] Ultralytics, "Yolov5," [Online]. Available: <https://github.com/ultralytics/yolov5>
- [44] G. Baulig, T. Gulde, C. Curio, "Adapting Egocentric Visual Hand Pose Estimation Towards a Robot-Controlled Exoskeleton," in *Proc. European Conference on Computer Vision (ECCV)*, Munich, Germany, 2018, pp. 241-256.
- [45] C. Qian, X. Sun, Y. Wei, X. Tang and J. Sun, "Realtime and Robust Hand Tracking from Depth," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 1106-1113.
- [46] X. Sun, Y. Wei, Shuang Liang, X. Tang and J. Sun, "Cascaded hand pose regression," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 824-832.
- [47] China Technical Committee for Ergonomics Standardisation. 3rd Congress. (2023, Aug. 6). *GB/T 10000-2023, Human dimensions of Chinese adults*. [online]. Available: <https://openstd.samr.gov.cn/bzgk/gb/newGbInfo?hcno=B19DCCA575D9406856ABF87A511EE11F>
- [48] Y. Yan *et al.*, "Survey on applications of algebraic state space theory of logical systems to finite state machines," *Sci. China-Inf. Sci.*, vol. 66, 2023, Art. no. 111201.